

Projekt ScrLk



Röststyrd AI-assistent via analog telefon på Macintosh Classic II med Raspberry Pi 5

Författare: Edward Falk

Företag: EQ2

Handledare (företag): Johannes Wiig

Handledare (skola): Henrik Hallenberg

Examinator: Henrik Hallenberg

Utbildning: Elektronikingenjör – inbyggda system (Ei24), Yrgo

Datum: 2026-05-19

Sammanfattning

Examensarbetet beskriver hur en Macintosh Classic II från 1991 får liv på nytt som ett modernt röststyrt AI-system. Det ursprungliga moderkortet har ersatts av en Raspberry Pi 5 som via DPI/KMS driver originalets monokroma 9-tums CRT vid 512×342 pixlar, och en svensk Ericofon (kobratelefon) från 1950-talet fungerar som mikrofon och högtalare mot en molnbaserad AI-modell (Google Gemini). Arbetet har omfattat idrifttagning av en 35 år gammal analog CRT-skärm med CRT-typiska 15 kV-risker, modifiering av en analog kolmikrofonkrets för A/D-omvandling, brygga mellan Apple Desktop Bus (ADB) och USB via en mikrokontroller, samt utveckling av ett strikt monokromt 1-bitars användargränssnitt som efterliknar det klassiska Macintosh-utseendet. Resultatet är ett fungerande system där användaren kan lyfta luren på en kobratelefon, ställa en fråga och få svar både i tal och i skrift på skärmen. Rapporten redogör för tekniska val, mätningar, problem under idrifttagningen och förbättringsförslag.



Figur 1: Projekt ScrLk i drift – en Macintosh Classic II integrerad med en Ericofon.

Sammanfattning.....	2
1. Inledning.....	5
1.1 Bakgrund.....	5
1.2 Företaget EQ2.....	8
1.3 Syfte och mål.....	8
1.4 Problemformulering.....	8
1.5 Avgränsningar.....	9
2. Metod.....	9
3. Systemöversikt.....	10
4. Hårdvara.....	10
4.1 Macintosh Classic II — analogkortet och CRT.....	10
4.2 Anslutning Pi 5 → analogkortet via Molex 14-pin.....	11
4.3 Spänningsnivåer och level shifter.....	12
4.4 DPI/KMS-konfiguration på Raspberry Pi 5.....	12
4.5 Ringsignalsgenerator.....	13
4.6 Ericofon — kolmikrofon och USB-ljudkortsmodifiering.....	13
4.7 Apple Extended Keyboard via ADB → USB.....	14
4.8 Externa instrument och mätarbete.....	14
5. Mjukvara.....	14
5.1 Operativsystem och desktop-stack.....	14
5.2 Happy Mac-UI och 1-bits rendering.....	14
5.3 Backend-API.....	15
5.4 AI-integration via Gemini Live.....	15
5.5 Wake-word.....	15
5.6 Deploy och övervakning.....	16
6. Resultat.....	16
6.1 Sammanfattat utfall.....	16
6.2 Verifiering.....	17
7. Diskussion.....	18
7.1 Måluppfyllelse vs projektplan.....	18
7.2 Tekniska lärdomar.....	18
7.3 Förbättringsförslag.....	18
7.4 Relevans för yrkesrollen.....	18
8. Slutsats.....	19
9. Referenser.....	19
10. Appendix.....	20
A. Komponentlista (BoM).....	20
B. DPI-overlay-konfiguration.....	20
C. Kodförråd.....	20
D. Inspirationskällor och relaterat arbete.....	21
E. Bilder och mätprotokoll.....	21

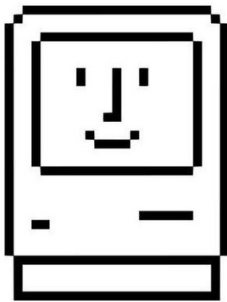
1. Inledning

1.1 Bakgrund

Under min första LIA-period på EQ2 i Göteborg hösten 2025 fick jag och min studiekamrat Felix Gunnarsson ett ovanligt uppdrag av Johannes Wiig på företaget. Han hade köpt en trasig Macintosh Classic från 1991 och en svensk Ericofon (även kallad "kbratelefon") från 1950-talet, och formulerade uppgiften ungefär så här:

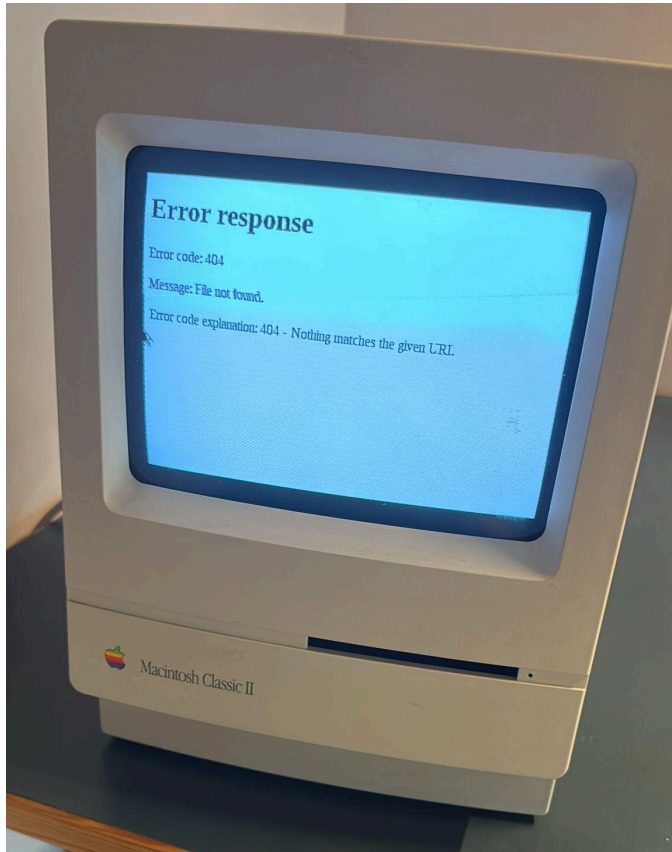
"Jag vill kunna prata med datorn genom telefonen."

Projektformuleringen var medvetet öppen. Hur det skulle göras var upp till oss. Projektet döptes till ScrLk (efter Scroll Lock-tangenten — en knapp som nästan ingen längre vet vad den gör, vilket kändes passande för ett projekt vars syfte var att ge ny mening åt något bortglömt) och började kartlägga vad som behövdes. AI:n i systemet kom att döpas Happy Mac, efter den glada pixliga ikonen som Susan Kare designade åt Apple.



Figur 2: Susan Kares ikon "Happy Mac", den visuella förebilden för systemets AI-persona.

Under LIA 1 byggde vi en första prototyp. Vi kombinerade chassit och bildröret från en någorlunda välbehållen Macintosh Classic med interna komponenter från en Macintosh Classic II, ersatte det trasiga moderkortet med en Raspberry Pi 5, kopplade ihop Pi:n med analogkortet så att den kunde driva originalets CRT, byggde en webbaserad simulerad operativsystemsmiljö som körs i en Chromium-kiosk, och kopplade in Ericofon-luren via ett modifierat USB-ljudkort. Vi visade upp prototypen i skolan efter LIA 1, men under större delen av redovisningen fungerade systemet dåligt — efter någon minut blev skärmen svart. När den framåt slutet av presentationen vaknade igen så funkade inte spelen jag gjort som de skulle. Men det gick att prata med AI:n Happy Mac genom kbratelefonen!



Figur 3: En 404-felkod på den monokroma skärmen.

Den lustiga första idén hade vuxit till ett system som kombinerade analog elektronik från fyra olika decennier med moderna molntjänster, jag hade gjort en svartvit version av Doom, en AI-styrd variant av Zork och början till ett litet operativsystem för datorn, och det blev tydligt att projektet förtjänade en ordentlig genomgång. När examensarbetet närmade sig hade jag egentligen tänkt göra ett helt annat projekt tillsammans med Felix, men jag kunde inte släppa ScrLk och Happy Mac. Jag valde att fortsätta med ScrLk och göra det till en stabil, demonstrationsbar produkt. Felix valde att gå vidare med projekt EQ200I (passiv kartläggning av industriell nätverkstrafik), men vi planerar en brygging mellan systemen där Happy Mac kan svara på frågor om data som Felix verktyg samlar in.

Tidslinje i korthet

| Datum | Händelse |

| --- | --- |
| 2025-10-23 | Felix dokumenterar LIA 1-chassiarbetet på video. |
| 2025-11-14 | Repostart för versionen av projektet som gäller idag. Första commit. |
| 2026-01-14 | Projektgrupp "ScrLk" etableras formellt. |
| 2026-02-07 | Johannes börjar lägga upp peppande inspirationsmaterial i chatten. |
| 2026-02-08 | Kompletta omtag på Happy Mac-UI; ny webbapp. |
| 2026-02-10 | Online-prototyp av Happy Mac live. |
| 2026-03-17 | Kombinerad demo med Doom, Zork, kontrollpanel, AI-chatt och Comm Log. Stabilt. |
| 2026-03-18 | Cloudflare-tunnel för möjlighet till samtal med HM från mobil |
| 2026-03-24 | Happy Mac får en egen gmail |
| 2026-04-14 | Jag uttryckte nöjdhet med att allt var så stabilt varpå allt slutade funka, varje fix av ett problem skapade två nya. Kod fullständigt kaotisk av alla nya funktioner jag lagt in |
| 2026-05-07 | Saker fungerar bättre igen, men man får obehagligt skarpa stötar om man nuddar ljudkortet, ljud slutar fungera. Planering av hårdvaruförbättringar. |
| 2026-05-19 | Inlämning slutrapport. |

1.2 Företaget EQ2

EQ2 är en konsultfirma i Göteborg med fokus på elektronik och inbyggda system. 10 % av årlig vinst doneras till Majblomman och Stadsmissionen. Företaget har en nyetablerad "inhouse"-satsning där man tar in egna projekt vid sidan av kunduppdrag. ScrLk kan kanske sägas vara ett sådant inhouse-projekt.

1.3 Syfte och mål

Syftet med examensarbetet är att utveckla en stabil version av ScrLk-systemet och dokumentera de elektroniska och systemintegrationsmässiga utmaningarna i att koppla ihop modern inbyggd hårdvara (Raspberry Pi 5) med 35 år gammal analog elektronik (CRT-skärm, Ericofon-mikrofon, ADB-tangentbord), så att slutanvändaren får en autentisk Macintosh Classic-upplevelse med tillgång till en modern röst-AI.

Konkreta mål:

- * Driva originalets 9-tums monokroma CRT från en Raspberry Pi 5 via DPI/KMS i 512×342 pixlar vid 60 Hz, utan flimmer eller geometriska artefakter.
- * Använda Ericofonens kolmikrofon och högtalare som ljudingång och ljudutgång för kommunikation med en molnbaserad AI-modell.
- * Möjliggöra inmatning från Apple Extended Keyboard (ADB-protokoll, 1990) via en USB-brygga.
- * Erbjud ett 1-bitars användargränssnitt som upplevs som en autentisk Macintosh-miljö.
- * Säkerställa att systemet startar av sig självt och kan demonstreras pålitligt under en hel presentation.

* Dokumentera samtliga elektriska gränssnitt, spänningsnivåer, signaltimings och säkerhetsåtgärder så att andra kan reproducera bygget.

1.4 Problemformulering

Hur kan en modern inbyggd Linux-plattform (Raspberry Pi 5) integreras med 35 år gammal analog elektronik (Macintosh Classic II:s CRT-bildrör, Ericofonens kolmikrofon, Apple Desktop Bus-tangentbord) så att användaren upplever en autentisk Macintosh Classic-dator som man förvånande nog kan föra ett röstsamtal med, och vilka är de tekniska valen och mätningarna som krävs för att överbrygga skillnaderna i spänningsnivåer, signaltiming och protokoll mellan epokerna?

1.5 Avgränsningar

Arbetet avgränsas från följande:

- * Egen AI-modell — Google Gemini används som molntjänst för språkförståelse och svarsgenerering.
 - * Restaurering av originalets digitala moderkort — det gamla moderkortet undersöks endast i den mån det behövs för att förstå signalgränssnittet mot analogkortet.
 - * Långtidsstabilitet och produktionsfärdighet — systemet är ett demonstrationsobjekt, inte en kommersiell produkt.
 - * Krav på EMC- och CE-certifiering hanteras inte.
-

2. Metod

Arbetet har följt en iterativ utveckling där jag växlat mellan att lösa hårdvarufrågor, mjukvarufrågor och systemintegration. Det mesta av hårdvarudelen löstes under LIA 1 när även Felix var med. Under LIA 2-perioden har det varit nästan enbart programmering och felsökning.

Forensisk elektronik. För Macintoshen var publika resurser tillräckliga som utgångspunkt — främst Bob Paradisos schematics-PDF för Mac Classic II [8] och bitsavers.org-arkivet [5] — men den dokumentation som faktiskt avgjorde pinouten kom från egna mätningar (huvudsakligen utförda av Felix Gunnarsson med multimeter och oscilloskop på det analoga kortet). För Ericofonen var situationen annorlunda: dess interna kretsdokumentation är i princip obefintlig online och fick istället hämtas från pärmar från 1970- och 1980-talen på Radiomuseet i Göteborg.

Litet, mätbart, ofta. Varje delsteg har verifierats fysiskt innan nästa byggts ovanpå. Spänningar har mätts före varje inkoppling till Pi:n eftersom ett misstag på 12 V-matningen kan döda en Pi 5 omedelbart (vilket vi lärde oss av erfarenhet). Signaltimingar (Hsync, Vsync, pixelklocka) har verifierats på oscilloskop innan KMS-drivrutinen aktiverats.

Konfiguration framför specialhårdvara. I stället för att designa eget PCB har vi använt befintliga moduler (level shifter, DC-DC, USB-ljudkort, Arduino Uno) och löst integrationen i mjukvara och konfigurationsfiler där det är möjligt. Det har hållit nere komplexiteten och gjort systemet enklare att felsöka hittills. Nu är dock systemet väldokumenterat nog för att eget PCB skulle kunna vara en bra idé.

AI-assisterad utveckling. Stora delar av mjukvaran har skrivits med hjälp av Claude Code. Det har snabbat upp arbetet betydligt på kodsidan, men har inte ersatt det grundläggande hårdvaru- och systemarbetet, som krävt fysisk närvaro, mätinstrument och iterativ felsökning.

3. Systemöversikt

Systemet består av tre fysiska huvuddelar och en molntjänst:

1. Macintosh Classic II-chassit med originalets analogkort och CRT, samt dess högspänningskrets.
2. Raspberry Pi 5 som ersätter originalets digitala moderkort. Pi:n driver CRT:n direkt via Display Parallel Interface (DPI) och kör hela mjukvarustacken.
3. Externa periferier — Ericofon-telefon, Apple Extended Keyboard och Macintosh-mus, alla anpassade för att fungera som indata till Pi:n.
4. Google Gemini som molnbaserad AI-modell, anropad över internet via Gemini Live API.

Kopplingsscheman finns i bilaga B. En illustrerad, löpande underhållen variant av systemöversikten med fler bilder och video finns i ``docs/02-architecture.md`` i det publika dokumentationsrepot Project-ScrLk (se Appendix C).

4. Hårdvara

Hårdvaran är rapportens tyngdpunkt eftersom den utgör de elektroniskt mest krävande delarna av arbetet: brygga mellan epoker med olika spänningsnivåer, olika signaltimings och olika protokoll.

4.1 Macintosh Classic II — analogkortet och CRT

Macintosh Classic II (1991) består fysiskt av två stora kretskort i ett plastchassi: ett digitalt logikkort ("moderkortet") och ett analogkort. Originallets moderkort har en Motorola 68030-processor med tillhörande RAM, ROM och I/O, och kommunicerar med analogkortet via en Molex-kontakt med 14 positioner. Analogkortet ansvarar för:

- * Strömförsörjning (+12 V, -12 V, +5 V) till hela systemet.
- * Drivning av CRT-bildröret, inklusive den högspänning på ungefär 15 kV som styr elektronkanonens accelerationsspänning.
- * Vertikal- och horisontaldeflektion av elektronstrålen.
- * Förstärkning av videosignalen som modulerar elektronstrålens intensitet.

För examensarbetet behåller jag analogkortet intakt. Det jag ersätter är det digitala moderkortet: i stället för att en 68030-processor genererar Hsync, Vsync och videosignal, är det nu Raspberry Pi 5 som matar in dessa signaler i Molex-kontakten.

Säkerhetsåtgärder. Innan jag arbetade med analogkortet utförde jag följande:

- * Datorn fick stå strömlös i minst 30 minuter för att låta primärkondensatorerna självurladda.
- * Anodkåpan urladdades aktivt genom att jordad mätledning fördes in under gummiproppen och kontakt etablerades mot anodanslutningen.
- * Multimeter användes för att verifiera att inga större kondensatorer fortfarande höll spänning.
- * Visuell inspektion efter "Mac-döden" (läckande elektrolytkondensatorer).

Misstänkta kondensatorer — och varför vi ändå inte bytte ut några. Under tidiga drifttester noterades flera symptom som klassiskt brukar peka mot defekta elektrolytkondensatorer: en distinkt lukt, en fladdrig bild, långa perioder med helt svart skärm och tvärt avbruten ström. Innan bytet hann göras upptäcktes dock att en mjukvarubugg fick en av Pi:ns CPU-kärnor att gå konstant på 100 % belastning (en okontrollerad renderingsloop, se 5.6). När den buggen var rättad försvann samtliga av de instabila symptomen. Eftersom byte av SMD-elektrolyter innebär reell risk, fattades beslutet att avvakta. Lukten är inte definitivt förklarad, men hypotesen att den kom från en torr eller marginellt läckande kondensator som inte ger elektriska problem under nuvarande driftsbild kvarstår som rimligast.

4.2 Anslutning Pi 5 → analogkortet via Molex 14-pin

Originallets moderkort kommunicerar med analogkortet via en 14-positioners Molex-kontakt.

Pin	Signal	Riktning från Pi	Anmärkning
---	---	---	---
1	+12 V	in	Försörjning från analogkortet
2	+12 V GND	—	

3	-12 V in	Försörjning från analogkortet
4	+5 V in	Försörjning från analogkortet
5	+5 V GND —	
6	Audio ut	Analog ljudsignal till intern högtalare
7	Audio GND —	
8	Video ut	1-bit monokrom videosignal
9	Video GND —	
10	Hsync ut	Horisontalsynk (~22 kHz)
11	Vsync ut	Vertikalsynk (60 Hz)
12	Hsync GND —	
13	(reserverad) —	
14	(reserverad) —	

Denna pinout bygger på publika scheman för Macintosh-familjen, men har empiriskt bekräftats vara korrekt i och med att systemet nu fungerar stabilt. Av särskild vikt är att pinnarna ligger fysiskt tätt och en kortslutning mellan exempelvis pin 1 (+12 V) och pin 8 (Video) skulle förstöra Raspberry Pi:ns GPIO direkt.

4.3 Spänningsnivåer och level shifter

Raspberry Pi 5:s GPIO-pinnar arbetar på 3,3 V CMOS-nivå. Det ursprungliga Macintosh-moderkortet arbetade i TTL-domän med signaler nominellt vid 5 V. För att skydda Pi:ns ingångar och säkerställa att Pi:ns utgångar tolkas korrekt av analogkortet används en bidirektionell level shifter. En DC-DC-omvandlare används för att alstra Pi:ns 5 V-matning från analogkortets +12 V-skena.

Identifierat problem — underdimensionerad level shifter. Den level shifter som monterades under LIA 1 har visat sig vara underdimensionerad. Den specificeras till maximalt 3 A genomgångsström och har dessutom en switchhastighet som ligger nära gränsen för Pi:ns DPI-pixelklocka. Symptomen har varit USB-over-current-varningar och spontant flimmer vid hög belastning. Komponentbytet är planerat men har inte hunnit utföras än.

Mätning av spänningsnivåer. Innan Pi:n kopplades in mättes spänningarna på Molex-kontakten med analogkortet i drift. Mätningarna bekräftade att de spänningsnivåer Pi:n exponeras för är inom säkra gränser efter level shifter-passering.

4.4 DPI/KMS-konfiguration på Raspberry Pi 5

Raspberry Pi 5 har ingen analog videoutgång. Lösningen är Display Parallel Interface (DPI), som lägger ut råa pixeldata och synksignaler direkt på Pi:ns GPIO-pinnar. DPI-läget aktiveras via en device-tree-overlay (`classic-crt-dpi-overlay.dts`).

Tidsdiagram.

*Synlig upplösning:** 512 × 342

*Pixelklocka:** 15,6672 MHz

*Horisontal blanking:** 195 pixlar

*Hsync-frekvens:** 22,158 kHz

*Vertikal blanking:** 29 rader

*Vsync-frekvens:** 59,726 Hz

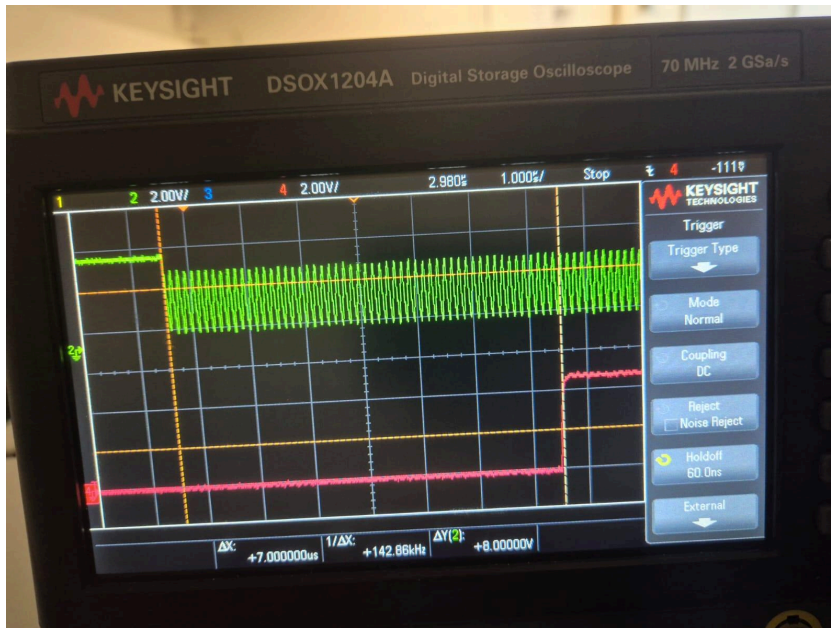
*Synkpolarietet:** Aktiv LOW på både Hsync och Vsync

*Bus-format:** 0x2001 (Y8 grayscale)

Två detaljer förtjänar att lyftas:

1. **Y8 grayscale i stället för RGB565:** Detta halverar DPI-bandbredden och ger en ärligare bild, eftersom frontenden redan levererar en strikt 1-bitars rendering.
2. **Aktiv LOW-synk:** Båda synkpulserna är inverterade jämfört med VESA-konventionen, vilket matchar analogkortets förväntningar.

Hsync-, Vsync- och pixelklocka från Pi:n verifierades med ett Keysight DSOX1204A digitalt oscilloskop och dokumenterades visuellt (se Figur 4 och Appendix D).



Figur 4: Verifiering av pixelklocka och synksignaler med Keysight-oscilloskop.

4.5 Ringsignalsgenerator

Som en utökning av luren roll har Felix Gunnarsson utvecklat en separat krets som genererar en ringsignal (25 Hz, ~70 V_{AC}). Eftersom ringsignalen ligger på en spänningsnivå som skulle förstöra ett vanligt USB-ljudkort kompletteras kretsen med ett mekaniskt relä.

4.6 Ericofon — kolmikrofon och USB-ljudkortsmodifiering

Ericofonen innehåller en kolmikrofon (carbon microphone) som kräver en konstant likspänningsmatning genom en bias-resistor, och ett dynamiskt högtalarelement. För att kunna föra ljud in och ut till Raspberry Pi:n modifierades ett standard-USB-ljudkort:

- * Mikrofonkretsen utökades med en bias-resistor och kondensatorkopplad ingång.
- * Högtalarutgången kopplades till Ericofonens högtalarelement.

4.7 Apple Extended Keyboard via ADB → USB

Apple Extended Keyboard kommunicerar över Apple Desktop Bus (ADB). För att låta tangentbordet fungera med Raspberry Pi:n byggdes en brygging baserad på en Arduino Uno R3. Data-pinnen är ansluten till Arduinons GPIO 8 och dras till +5 V via ett 1 kΩ pull-up-motstånd. ATmega328P läser av pinnen med tajming på mikrosekundnivå.

Firmwaren bygger på ett publikt tillgängligt projekt för ADB-till-USB-konvertering.

4.8 Externa instrument och mätarbete

Följande instrument har använts:

- * Digital multimeter för spännings- och kontinuitetsmätningar på Molex-kontakten, ADB-dataledningen och Ericofonens kolmikrofon.
- * Keysight DSOX1204A digitalt minnesoscilloskop (70 MHz, 2 GSa/s).
- * Lödstation och avlödningsverktyg.

Instrumenten har använts kontinuerligt under hela idrifttagningen för att validera signalkedjan från Pi till CRT, samt audiogenomströmningen genom Ericofonen.

5. Mjukvara

Endast huvuddragen presenteras; detaljer finns i `docs/`-katalogen i kodförrådet.

5.1 Operativsystem och desktop-stack

Pi:n kör Raspberry Pi OS Lite (64-bit). Ovanpå detta läggs *labwc* (minimal Wayland-kompositor), *lightdm* och *Chromium* i kiosk-läge.

5.2 Happy Mac-UI och 1-bits rendering

Webbappen "Happy Mac" är en React-applikation som efterliknar Mac System 6/7 med strikt 1-bitars rendering (endast helt svart eller helt vit). Detta säkerställs genom ingen alpha/opacity, ingen antialiasing, bitmap-fonter och server-side ditrerade färgbilder. Tre lager av regelkontroller används i koden för att undvika färg/gråskala-läckage i UI:t.

5.3 Backend-API

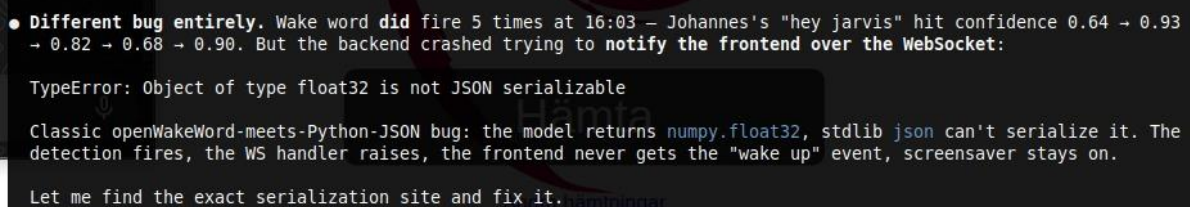
En FastAPI-baserad Python-backend hanterar systemstatus, spelstyrning, bildditrering och hälsoövervakning.

5.4 AI-integration via Gemini Live

Röst- och chattfunktionaliteten drivs av Google Gemini Live API (Gemini 3.1 Flash Live). En systeminstruktion ger modellen en personlighet i linje med Macintosh Classic-konceptet.

5.5 Wake-word

En lokal wake-word-modell (`openWakeWord`) lyssnar efter frasen "hey jarvis" (när detta fungerar stabilt är nästa steg att träna den att lyssna efter "hey mac" istället). Wake-word-modulen var komplex att stabilisera, särskilt för att hantera scenarion där USB-ljudkortet fysiskt dras ur under drift (se Figur 5 och 6).



```
● Different bug entirely. Wake word did fire 5 times at 16:03 – Johannes's "hey jarvis" hit confidence 0.64 → 0.93 → 0.82 → 0.68 → 0.90. But the backend crashed trying to notify the frontend over the WebSocket:

TypeError: Object of type float32 is not JSON serializable

Classic openWakeWord-meets-Python-JSON bug: the model returns numpy.float32, stdlib json can't serialize it. The detection fires, the WS handler raises, the frontend never gets the "wake up" event, screensaver stays on.

Let me find the exact serialization site and fix it. namlindgar
```

Figur 5: Felsökning wake-word confidence och tröskelvärden.

```
TL;DR
Johannes did show up in the log. His best attempt peaked at confidence 0.484 yesterday – threshold is 0.50. He was almost certainly speaking "hey jarvis" intelligibly; the audio path is corrupted, not the keyword model.

Evidence
GET /api/v1/audio/subsystem
→ audio backend: "ALSA"
→ pipewire connected: false
→ capture_devices: []
→ wake_word.listening: true ← service *thinks* it's listening

wpctl status confirms: zero Streams: under the USB mic (PCM2902 Audio Codec Analog Mono, source #48). Chromium is there as a PipeWire client, but the wake-word service isn't – it's off on an ALSA side path, probably fighting over the kernel PCM device and getting near-silence.

Root cause – regressed %U bug
```

Figur 6: OpenWakeWord gav massor av spännande möjligheter till nya problem

5.6 Deploy och övervakning

Systemet är provisionerat via Ansible. En övervakningsdaemon kontrollerar regelbundet att systemet mår bra.

Efter en okontrollerad renderingsloop (runaway render-loop) 2026-03-24, visade det sig att den aggregerade CPU-siffran Linux exponerar kan dölja en maxbelastad CPU-kärna (en kärna på 100 %, fyra på ~20 % blir 40 % i snitt). Som direkt följd lades en on-screen WORKSTATION-widget till i UI:t som visar enskilda kärnors belastning så att en mänsklig blick under drift omedelbart kan upptäcka en maxbelastad CPU-kärna.

6. Resultat

6.1 Sammanfattat utfall

Den slutgiltiga prototypen uppfyller samtliga mål:

- * CRT:n drivs stabilt vid 512×342 / 60 Hz utan flimmar eller geometriska artefakter.
- * Användaren kan lyfta luren på Ericofonen, ställa en fråga och få svar i tal genom luren samtidigt som svaret renderas i Happy Mac-UI:t.
- * Apple Extended Keyboard fungerar för all text-inmatning.
- * Vid uppstart bootar Pi:n direkt in i Happy Mac-UI:t.
- * Systemet kan demonstreras pålitligt under en längre presentation.



Figur 7: Slutresultatet – Happy Mac spelar Zork och genererar bilder till spelet live.

6.2 Verifiering

*Skärmrendering:** Verifierat med kamerafoto av CRT under drift.

*Signaltimingar:** Verifierat med oscilloskop under idrifttagning.

*AI-konversation:** Verifierat genom funktionstester över flera timmar.

*Spänningsnivåer:** Verifierat med multimeter; GPIO-ingångar exponeras aldrig för spänningar utanför 0–3,3 V.

7. Diskussion

7.1 Måluppfyllelse vs projektplan

Den ursprungliga projektplanen beskrev passiv kartläggning av industriell nätverkstrafik. Jag valde senare att i stället vidareutveckla ScrLk/Happy Mac. Den ursprungliga projektplanens mål kan således inte användas som direkt jämförelse mot utfallet, vilket i stället har stämts av löpande med handledare.

7.2 Tekniska lärdomar

Forensisk dokumentation (forensic documentation) är en yrkesfärdighet. Att växla mellan gamla ritningar, fysiska mätningar och online-källor — och dokumentera slutsatserna — är en kärnkompetens.

Strikt gräns mellan spänningsdomäner. Level shiftern som fysisk skiljelinje och att "mäta först, koppla sen" är avgörande principer för att inte bränna hårdvara.

Symptom respekterar inte domängränser. Mjukvarubuggar (renderingsloop) producerade en symptombild som såg ut som hårdvarufel (trasiga kondensatorer). Lågnivåövervakning är därmed kritisk för felsökning.

Ljud är svårare än bild. Den enskilt mest komplexa delen (förutom att hitta rätt typsnitt till OS:et, det tog mig veckor ...) att stabilisera var ljudkedjan med dess konkurrerande drivrutiner (se Figur 6) och den analoga kolmikrofonen.

*Molnleverantörers API-kontrakt kan ändras tyst.** Gemini 3.1 ändrade beteende utan synlig versionsbump. Detta belyser vikten av prober som testar faktiskt beteende i systemet.

7.3 Förbättringsförslag

Byt level shifter: Till en korrekt dimensionerad komponent med högre strömkapacitet och snabbare switchhastighet.

Eget interface-PCB: Ett dedikerat kretskort för bryggan skulle minska risken för kortslutning.

Galvanisk isolation: På audio-vägen för att få bort brus från digital-sidan.

*I2S-DAC:** Byta USB-ljudkortet mot en dedikerad I2S-lösning för lägre latens.

Egen wake-word-modell: Tränad på svensk fonologi.

7.4 Relevans för yrkesrollen

Arbetet har övat systemintegration mellan epoker, forensiska mätningar, säkerhet kring högspänning och värdet av strukturerad dokumentation — kompetenser som är direkt överförbara till arbete med industriella styrsystem.

8. Slutsats

Examensarbetet har visat att det är fullt möjligt att kombinera en modern inbyggd Linux-plattform (Raspberry Pi 5) med 35 år gammal analog elektronik så att resultatet upplevs som en sammanhållen, autentisk produkt. De tekniskt svåraste delarna var kopplingen mellan spänningsdomäner, signaltimings och protokoll — och i synnerhet stabiliseringen av ljudkedjan.

Den genomgående lärdomen är ändå att detaljerna man inte ens tänkte på ofta blir det största problemet, och att till synes oskyldiga extrafunktioner som att kunna väcka AI-chatten genom att säga Hey Mac i luren kan råka ta en ansenlig del av all tid man lägger på hela projektet. Scope creep är farligt, men ack så roligt.

9. Referenser

- [1] Yrgo. Studieplan i kursen Examensarbete 10 poäng. Rev. 2.0, 2026-05-04.
- [2] Yrgo. Kursplaner Elektronikingenjör inbyggda system. YH00057-2023-2.
- [3] Raspberry Pi Foundation. DPI configuration.
- [4] Raspberry Pi Foundation. Raspberry Pi 5 hardware reference.
- [5] Apple Computer Inc. Macintosh Family Hardware Reference. Addison-Wesley, 1988.
- [6] Apple Computer Inc. Apple Desktop Bus. I Inside Macintosh: Devices. Addison-Wesley, 1994.
- [8] Bob Paradiso. Macintosh Classic II — schematics och teknisk dokumentation.
- [9] Google. Gemini Live API documentation.
- [10] openWakeWord-projektet.

10. Appendix

A. Komponentlista (BoM)

Komponent	Beskrivning	Källa	Ungefärligt pris
---	---	---	---
Raspberry Pi 5 (8 GB)	Huvudplattform	Electrokit	1 500 kr
microSD 64 GB	OS-lagring	—	200 kr
Macintosh Classic II	Chassi + analogkort + CRT	Begagnat	—
Macintosh Classic	Reservdelar	Begagnat	—
Ericofon	Mikrofon + högtalare + gaffel-switch	Begagnat	—
USB-ljudkort	Audio in/ut	—	150 kr
Level shifter	3,3 V ↔ 5 V	—	100 kr
DC-DC 12 V → 5 V	Pi-matning	—	100 kr
Arduino Uno R3	ADB-USB-brygga	—	250 kr
Apple Extended Keyboard	Indata	Begagnat	—
Pull-up-resistor 1 kΩ	ADB-data → +5 V	—	<10 kr
Bias-resistor + kondensator	Kolmikrofonens bias-krets	—	<20 kr
Mekaniskt relä	Audio-kortets skydd under ringintervaller	Skolan	—
Transformatorer	Galvanisk koppling	—	—
Diverse kablage, kontakter	—	—	300 kr

B. DPI-overlay-konfiguration

Den fullständiga overlayen som driver Macintosh Classic-CRT:n från Pi 5 via DPI/KMS ligger i kodbasen som ``infrastructure/ansible/roles/classic_crt/files/classic-crt-dpi-overlay.dts`` och kompileras av Ansible-playbooken till ``dtbo`` vid provisionering.

C. Kodförråd

Project-ScrLk – Publik dokumentation:
<https://github.com/edwardfalk/Project-ScrLk>

D. Inspirationskällor och relaterat arbete

Följande projekt har studerats som referens för teknisk lösning eller för helhetskonceptet "ny hårdvara i gammalt chassi":

- * Frank Chiarulli — "Building the Rcade"
- * Duncan Hall — "Restomod" Apple Macintosh SE
- * likeablob/macmini
- * sakofchit/system.css — CSS-designsystem för retro Apple-gränssnitt

E. Bilder och mätprotokoll

Samtliga väsentliga bilder och diagram har integrerats löpande i rapporten som Figur 1 till 7. För ytterligare mätprotokoll och högupplösta videosekvenser från idrifttagningen, se dokumentationsrepet angivet i Appendix C.